

INDIVIDUAL SELF-LEARNING ACTIVITY | TASK 2

INTERACTIVE JAVASCRIPT & REACTJS APPLICATION DEVELOPMENT

TOPIC 6: ONLINE QUIZ APPLICATION

Comprehensive System Architecture, Implementation & Slide Deck Report

Course:	Full Stack Web Development
Problem Assigned:	No. 6 - Online Quiz Application
Student Name:	KIRANKUMAR G
Class & Section:	58 - III CSE - A Section
Assessment Date:	October 2026
Submission Type:	ReactJS Source Code, Report & Slides

Mandatory Submission: GitHub Repository Link Included • Live Demo Enabled

1. Executive Summary & Problem Formulation

Overview of Problem Statement 6 and Course Objectives

As part of the Full Stack Web Development Individual Self-Learning Activity (Task 2), this project implements Problem Statement No. 6: Online Quiz Application. The objective is to architect and develop an interactive, highly responsive, and robust ReactJS application that evaluates candidates via multiple-choice quizzes, featuring real-time countdown timers, dynamic score calculation, answer validation, detailed result analysis, and quiz management.

1.1 Problem Statement Definition (PDF Reference)

Assigned Statement No. 6: Online Quiz Application

"Create a multiple-choice quiz with score calculation, answer validation, and a result page."

1.2 Compliance with Common Requirements

- 1. Interactive & Responsive ReactJS App:** Crafted with component-based modular structure and reactive UI optimized across mobile, tablet, and desktop viewports.
- 2. Modern HTML5 & CSS3 Design System:** Engineered with glassmorphism, responsive CSS grid, typography, custom badges, and seamless Dark/Light theme switching.
- 3. React Components, JSX & Modern Hooks:** Utilizes useState for dynamic user selections, time management, and view routing; useEffect for timer countdowns and storage synchronization.
- 4. Form Handling & Client-Side Validation:** Provides a custom Quiz Creator form with validation for title length, required category, duration limits (1-60 mins), and 4 complete options.
- 5. Interactive Features (CRUD, Search, Filter, Calculate):** Supports adding custom quizzes, editing, deleting, live text search, category filtering, difficulty selection, and real-time grade calculations.
- 6. Multiple Functional Pages / Views:** Contains 5 interconnected views: (1) Browse Quizzes, (2) Active Quiz with Timer, (3) Detailed Results & Solutions, (4) Quiz Creator, and (5) History.
- 7. Project Organization & Meaningful Naming:** Organized under clean directories: src/components/, src/data/, and modular styling tokens.

2. System Architecture & Component Design

Modular structure, data flow, and state hierarchy

The application follows a unidirectional data flow pattern centered around top-level state managed within App.jsx. Persistent data (custom quizzes, attempt history, theme preference) is synchronized with browser localStorage.

2.1 Component Breakdown & Responsibilities

Navbar.jsx	[Global Navigation & Preferences]
Handles navigation between views, displays attempt count badge, and toggles dark/light visual mode.	
QuizList.jsx	[Home / Quiz Directory]
Presents available quizzes with category pills, difficulty filters, keyword search, score badges, and action buttons.	
QuizTake.jsx	[Interactive Test Interface]
Manages dynamic countdown timer, jump-to-question navigation pills, answer selection, question flagging, and submit confirmation.	
QuizResult.jsx	[Score & Educational Review]
Computes total score, percentage, and letter grade. Provides full question-by-question review with explanations and filter tabs.	
QuizManage.jsx	[Quiz Creator & Form Validator]
Form interface allowing teachers/students to author custom quizzes with strict client-side validation on every field.	
QuizHistory.jsx	[Historical Attempt Analytics]
Logs past attempts with timestamps, duration taken, percentage, grade, and allows viewing historical breakdowns.	

2.2 State Management & Hooks Strategy

- **useState Hook:** Manages activeView ("list" | "take" | "result" | "manage" | "history"), user-selected options ({ [qIdx]: optionIdx }), flagged questions, search queries, category filters, and active timer seconds.
- **useEffect Hook (Countdown Timer):** Implements a 1-second interval in QuizTake.jsx to decrement remaining time, cleanly clearing interval on unmount or on completion, and triggering auto-submission when timer reaches 00:00.
- **useEffect Hook (Persistence & Theming):** Automatically persists test attempts and custom quizzes to localStorage and syncs document data-theme attribute.

3. Application Workflow & Key Algorithms

Step-by-step assessment lifecycle and calculation algorithms

3.1 User Workflow & Lifecycle Stages

Step 1: Exploration & Selection — User filters quiz topics by category (ReactJS, JavaScript ES6, Web Dev, CS) or searches by keywords; selects difficulty and clicks Start Quiz.

Step 2: Assessment Session — QuizTake mounts: Countdown timer initiates; candidate selects radio choices; can flag questions for later review; uses question drawer to jump between questions.

Step 3: Submission Verification — Submit trigger checks total answered vs unanswered vs flagged. Shows confirmation modal with clear warnings if questions were omitted.

Step 4: Score Computation — Algorithm evaluates answers against correctAnswer indexes, computes percentage, calculates letter grade (O, A+, A, B, C, F), and saves record.

Step 5: Results & Answer Key Review — QuizResult displays performance summary, celebration animation, and color-coded answer review (green for correct, red for incorrect) with educational explanations.

3.2 Mathematical Score & Grade Algorithm

```
// Mathematical Score Calculation in App.jsx
let score = 0;
quiz.questions.forEach((q, idx) => {
  if (userAnswers[idx] === q.correctAnswer) {
    score += 1;
  }
});
const totalQuestions = quiz.questions.length;
const percentage = Math.round((score / totalQuestions) * 100);

// Academic Grade Assignment
let grade = 'F';
if (percentage >= 90) grade = 'O'; // Outstanding
else if (percentage >= 80) grade = 'A+'; // Excellent
else if (percentage >= 70) grade = 'A'; // Very Good
else if (percentage >= 60) grade = 'B'; // Good / Pass
else if (percentage >= 50) grade = 'C'; // Average
const passed = percentage >= 60;
```

4. PowerPoint Presentation Deck (Slides 1–4 of 7)

Mandatory 5-8 slides summary formatted for evaluation

Slide 1: Project Title & Introduction

Task 2 Presentation

- Title: Online Quiz Application (Problem Statement #6)
- Course: Full Stack Web Development (Assessment Date: 1.10.2026)
- Student: KIRANKUMAR G | III CSE - A Section
- Technology Stack: ReactJS, Vite, JavaScript ES6+, HTML5 & Modern CSS3
- Goal: Provide an intuitive, real-time assessment platform with live score validation

Slide 2: Objectives & Functional Scope

Architecture

- Multiple-choice quiz interface with randomized & categorized test sets
- Live dynamic countdown timer with automatic test submission upon expiration
- Instant score evaluation, percentage calculation, and academic grade assignment
- Detailed answer key validation showing student selection vs correct option
- Quiz creation & management with rigorous client-side form validation

Slide 3: Technology Stack & Technical Foundation

Core Stack

- React Components & JSX: Modular, reusable functional components
- React Hooks: useState for multi-view state; useEffect for timer countdowns
- Responsive CSS3: Fluid card glassmorphism, responsive grid layouts, and Dark/Light themes
- Client-Side Storage: Browser localStorage for persisting custom quizzes and attempt histories
- Modern ES6+: Destructuring, arrow functions, template literals, and array methods

- View 1 - Browse Quizzes: Search by keyword, filter by category/difficulty, view stats
- View 2 - Interactive Quiz: Timer badge, question navigation drawer, option selector, flagging
- View 3 - Results & Solutions: Score breakdown, grade badge, question-by-question explanations
- View 4 - Quiz Creator: Form validation for title, categories, options, and explanations
- View 5 - Attempt History: Timestamped log of past attempts with score records

5. PowerPoint Presentation Deck (Slides 5–7) & Testing

Slide continuation, test matrix, and evaluation criteria

Slide 5: Score Calculation & Answer Review	Evaluation Engine
• Automatic score calculation based on strict answer key comparison • Real-time calculation: $\text{Score} = \text{sum}(\text{isCorrect})$, $\text{Percentage} = (\text{score} / \text{total}) * 100$ • Letter Grades: O ($\geq 90\%$), A+ ($\geq 80\%$), A ($\geq 70\%$), B ($\geq 60\%$), C ($\geq 50\%$), F ($< 50\%$) • Review filter tabs: View All, Correct Only, Incorrect Only, or Skipped Questions • Comprehensive educational explanations provided for every single question	
Slide 6: Form Handling & Client-Side Validation	Quality & Security
• Title Validation: Enforces minimum 5 characters to ensure descriptive naming • Duration Control: Restricts quiz duration strictly between 1 and 60 minutes • Option Completeness: Validates all 4 options (A-D) and requires a correct answer selection • Explanation Requirement: Mandates solution rationale before allowing quiz publication • Inline Error Feedback: Visual red warning borders and descriptive error banners	
Slide 7: Conclusion & Key Learnings	Summary
• Successfully delivered a production-ready ReactJS Online Quiz Application • Mastered React hooks (useState, useEffect) for state synchronization and timers • Implemented clean, accessible, and responsive user interfaces with Dark/Light themes • Fulfilled 100% of Task 2 requirements: source code, report, slides, and GitHub repository	

5.1 Test Cases & Quality Verification Table

TC-01: Countdown Timer Auto-Submit	Timer decrements each second; triggers automatic submit at 00:00	PASSED
TC-02: Score & Grade Calculation	Accurate percentage and grade (O, A+, A, B, C, F) generated	PASSED
TC-03: Client-Side Form Validation	Prevents empty fields, requires 4 options and valid duration	PASSED
TC-04: Question Flagging & Navigation	Direct question jump and review flagging persist during test	PASSED
TC-05: Dark/Light Mode Switcher	Smooth theme transition across all components without layout shift	PASSED

6. Submission & GitHub Repository Details

Mandatory submission links and execution instructions

In adherence with Section 2 ("Submission Requirements") of the assignment guideline, all source code, documentation, and assets are version controlled with Git.

GitHub Repository Details:

Repository Name: online-quiz-application

Owner / Account: KIRANKUMAR G (kirang151205)

Repository URL: <https://github.com/kirang151205/online-quiz-application>

6.1 Quick Start & Demonstration Instructions

```
# 1. Clone the repository
git clone https://github.com/kirang151205/online-quiz-application.git

# 2. Navigate to project root
cd online-quiz-application

# 3. Install dependencies
npm install

# 4. Start local development server
npm run dev
```

```
# 5. Access in browser
```

6.2 Student Declaration & Verification

I hereby declare that this assignment "Task 2: Interactive JavaScript and ReactJS Application Development - Problem No. 6: Online Quiz Application" has been developed individually through self-learning concepts, utilizing React components, JSX, useState, useEffect, form handling, client-side validation, and modern CSS3 design principles.

Student Signature / Name: KIRANKUMAR G

Date: 04 October 2026